Shortest path

Dijkstra's algorithm

Michel Bierlaire

Optimization: principles and algorithms

ÉCOLE POLYTECHNIQUE
FÉDÉRALE DE LAUSANNE

Dijkstra's algorithm

In many applications, the cost on the arcs are all non negative.
In that case, the shortest path algorithm can be designed to be efficient
In particular, it is possible to guarantee that each node will be treated at most once.
This version of the algorithm is called Dijkstra's algorithm, from the name of Dutch researcher.

Non negative costs

Cycles

- No cycle with negative cost.
- No need to check if the problem is unbounded.

Node selection

Select the node in $\mathcal{S}$ with the smallest label.

Algorithm

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \; \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$ such that $\lambda_i \leq \lambda_j$, for all $j \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

Permanent labels

$$\mathcal{T} = \{i | \lambda_i \neq \infty \text{ and } i \notin \mathcal{S}\}.$$

At the end of each iteration

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \ \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$ such that $\lambda_i \leq \lambda_j$, for all $j \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If $i \in \mathcal{T}$ and $j \notin \mathcal{T}$, then $\lambda_i \leq \lambda_j$.

First iteration: o is the only node to be treated.
For all j updated: $\lambda_j = c_{oj} \geq 0 = \lambda_o$.
Iteration treating ℓ: assume property holds, and $\lambda_\ell \leq \lambda_j \forall j \notin \mathcal{T}$
Treat arc (ℓ, m), $m \in \mathcal{T}$:
$\lambda_m \leq \lambda_\ell + c_{\ell m}$. No update. MENTION THAT IT IS THE NEXT PROPERTY
arc (ℓ, m) $m \notin \mathcal{T}$: if updated
$\lambda_m = \lambda_\ell + c_{\ell m}$. As λ_ℓ was larger than any label in $\mathcal{T}$, the same holds for λ_m.
Finally, as ℓ joins $\mathcal{T}$, the property holds for it because of the node selection rule.

At the end of each iteration

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \ \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$ such that $\lambda_i \leq \lambda_j$, for all $j \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If $i \in \mathcal{T}$ at the beginning of the iteration, then the label λ_i is not modified during the iteration.

Was shown before

At the end of each iteration

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \ \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$ such that $\lambda_i \leq \lambda_j$, for all $j \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If $i \in \mathcal{T}$ at the beginning of the iteration, then $i \notin \mathcal{S}$ at the end of the iteration.

Corollary of the previous result

At the end of each iteration

Initialization

- $\lambda_o \leftarrow 0$,
- $\lambda_i \leftarrow +\infty \; \forall i \in \mathcal{N}, i \neq o$,
- $\mathcal{S} \leftarrow \{o\}$.

Choose $i \in \mathcal{S}$ such that $\lambda_i \leq \lambda_j$, for all $j \in \mathcal{S}$

$\forall (i,j)$, if $\lambda_j > \lambda_i + c_{ij}$

- $\lambda_j \leftarrow \lambda_i + c_{ij}$,
- $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$.

$\mathcal{S} \leftarrow \mathcal{S} \setminus \{i\}$

If $i \in \mathcal{T}$, then λ_i is the length of the shortest path from o to i.

Because the label is permanent. This is the interpretation at the end of the algorithm. The value will be the same

Summary

Dijkstra's algorithm is the generic shortest path algorithm where a rule is used for the selection of the node to be treated
Among the candidates in the set $\mathcal{S}$, we select the one with the smallest label
If all costs are non negative, this algorithm has the property that, once a node has been treated, its label is permanent. Therefore, each node will be treated at most one by the algorithm.